A PROJECT REPORT ON

STUDYCONNECT

A COLLEGE-FOCUSED ACADEMIC COLLABORATION PLATFORM

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted By

<< Name of the Student >>

<< Hall Ticket No >>

Under the Guidance of

<< Guide Name >>

<< Designation >>

<< College Logo >>

Department of Information Technology,

<<Your College Name in CAPS>>

Academic Year 2025-2026

CERTIFICATE

<<Your College Name in CAPS>>

Department of Information Technology

<< College Logo >>

XXXXXXXXX (11H61A12XX), XXXXXXXXX (11H61A12XX), XXXXXXXXXX (11H61A12XX) under the guidance of Mr./Ms/Mrs XXXXXXXXX, Designation

COMPANY CERTIFICATE

<< Company Certificate >>

DECLARATION

This is to Certify that the project work entitled “Name of the project submitted to JNTUH in partial fulfillment of the requirement for the award of the Degree of Bachelor of Technology (B-Tech), is an original work carried out by XXXXXXXXX (11H61A12XX), XXXXXXXXX (11H61A12XX), XXXXXXXXXX (11H61A12XX) under the guidance of Mr./Ms/Mrs XXXXXXXXX, Designation in the Department of Information Technology. This matter embodied in this project is a genuine work, done by the students and has not been submitted whether the university or to any other university/Institute for the fulfillment of the requirement of any course of study.

ACKNOWLEDGEMENT

The project team expresses its sincere gratitude to the project guide and the Department of Information Technology for their guidance and academic support during the development of StudyConnect. The team also acknowledges the faculty, reviewers, classmates, and family members whose feedback and encouragement supported the completion of this work.

ABSTRACT

StudyConnect is a college-focused collaboration platform that brings academic communities and student communication into one web and mobile experience. It provides account registration and authentication, student profiles, discoverable communities, community membership and moderation, real-time community chat, planned direct-messaging extensions, academic resource sharing, notifications, events, and administrative controls. The platform is organized as a multi-client system: a React web application and a React Native mobile client communicate with backend services through REST APIs and real-time channels.

The repository uses a single TypeScript and Node.js backend built with Express. Mongoose models store application data in MongoDB, while route, controller, middleware, service, and repository layers separate request handling from business rules and database access. Authentication, role and membership checks, request validation, security middleware, and structured error handling protect the API. Socket.IO provides community chat and other configured real-time events. File uploads pass through a storage service that supports local storage and configurable external providers. Provider credentials and production deployment settings must be supplied in the environment.

The project addresses fragmented academic coordination by providing a shared place for student communities, group communication, learning resources, and campus activity. It is designed to support modular growth while retaining account security, community membership rules, and administrative oversight. Future work includes deployment-readiness review, broader integration testing across clients, and continued improvements to accessibility, observability, and institutional onboarding.

Keywords: academic collaboration, student communities, real-time communication, Express, React, MongoDB, mobile application.

CONTENTS

List of Figures vi
List of Tables vi
List of Abbreviations vii
CHAPTERS **PAGE NO**
CHAPTER 1 INTRODUCTION 1
1.1 Overview
1.2 Objectives
1.3 Problem Formulation
1.4 Scope
1.5 Feasibility
1.6 System Requirements
CHAPTER 2 REQUIREMENTS 5
2.1 Requirements Specification
2.2 User Interfaces
2.3 Constraints and Prerequisites
CHAPTER 3 ANALYSIS 10
3.1 Existing System
3.2 Proposed System
3.3 Use Case Model
3.4 Flow of Events
3.5 Activity Flow
CHAPTER 4 DESIGN 13
4.1 System Architecture
4.2 Database Design
4.3 Entity Relationship Model
4.4 Sequence Design
4.5 Class and Responsibility Design
4.6 API and Security Design
CHAPTER 5 IMPLEMENTATION 18
5.1 Technology Stack
5.2 Modules
5.3 Implementation Details
5.4 Configuration and Deployment
CHAPTER 6 USER SCREENS 22
6.1 Navigation
6.2 Student Screens
6.3 Administration Screens
6.4 Mobile Screens
CHAPTER 7 TESTING METHODOLOGY 25
7.1 Testing Strategy
7.2 Representative Test Scenarios
7.3 Verification Status
CHAPTER 8 CONCLUSION AND FUTURE ENHANCEMENTS 27
8.1 Conclusion
8.2 Future Enhancements
CHAPTER 9 REFERENCES 28

LIST OF FIGURES

Figure No.	Description	Page No.
3.1	Primary use case model	10
3.2	Community message activity flow	12
4.1	Layered application architecture	13
4.2	Core entity relationship model	15
4.3	Community message sequence	16
4.4	Backend class responsibility model	16

LIST OF TABLES

Table No.	Description	Page No.
1.1	Software and hardware requirements	4
2.1	Functional requirements	5
2.2	Non-functional requirements	7
2.3	User interface inventory	8
4.1	Primary MongoDB collections	14
4.2	Roles and permissions	17
5.1	Technology stack	18
7.1	Representative test scenarios	26

LIST OF ABBREVIATIONS

Notation	Description
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
STOMP	Simple Text Oriented Messaging Protocol
UI	User Interface
UX	User Experience
WebSocket	Full-duplex communication protocol over a persistent connection

CHAPTER 1 INTRODUCTION

1.1 OVERVIEW

College learning involves more than course delivery. Students organize study groups, exchange notes, ask questions, coordinate events, and maintain peer connections. When these interactions are split across unrelated messaging and file-sharing tools, students have difficulty discovering relevant communities and locating useful academic material. StudyConnect provides a shared platform for these activities, with access governed by student accounts, community membership, and administrative roles.

The project is organized as a monorepo containing a React web application, a React Native mobile client, and a TypeScript/Node.js backend built with Express. Both clients communicate with the backend through HTTP APIs, and supported real-time features use Socket.IO. The backend is divided into routes, controllers, middleware, services, repositories, Mongoose models, and infrastructure providers. MongoDB stores accounts, communities, messages, and related records. This design keeps client screens separate from application rules and persistence operations.

1.2 OBJECTIVES

1. Provide authenticated student accounts and editable student profiles.
2. Enable students to create, discover, join, and manage academic communities with owner, moderator, and member roles.
3. Support authenticated community chat with persisted history and real-time updates, while treating participant-scoped direct messaging as a future extension.
4. Provide academic resource discovery and sharing, event participation, notifications, and student coordination features.
5. Provide administrators with user and content management capabilities.
6. Keep the application modular so web and mobile clients can use a common backend contract.

1.3 PROBLEM FORMULATION

Students often coordinate academic work through disconnected channels. This creates repeated effort in finding peers, sharing learning resources, and following group activity. The project problem is to design a secure, accessible system that combines community discovery, messaging, resource exchange, and campus activity while enforcing role-based access to student and administrative functions.

1.4 SCOPE

The project scope includes account registration and sign-in, student profiles, community discovery and membership, community chat, academic resources, notifications, administration, and client areas for events and study coordination. Access to protected information depends on authentication, community membership, recipient or participant identity, and administrative role checks. The web and mobile clients use the Express API and its supported real-time channels. Email, external storage, institutional identity, and other provider integrations require configuration and credentials; their availability depends on the selected deployment.

1.5 FEASIBILITY

Area	Assessment
Technical	The project uses established web, mobile, API, database, and real-time communication frameworks. The modular repository separates client and service concerns.
Operational	Students can access the platform through a browser or supported mobile build. Administrators manage accounts and reported content through protected routes.
Economic	The stack is based primarily on open-

Area	Assessment
	source frameworks. Hosting, database, object storage, email, and real-time service costs depend on selected providers and traffic.
Schedule	Work can be organized into feature milestones for identity, communities, communication, resources, administration, client integration, and deployment review.

1.6 SYSTEM REQUIREMENTS

1.6.1 Software Requirements

Component	Requirement / Technology
Web client	Node.js toolchain, React 19, TypeScript, Vite, React Router, Tailwind CSS
Mobile client	React Native 0.86, React Navigation 7, Android or iOS development environment
Backend service	Node.js 20+, TypeScript, Express 5, Mongoose, MongoDB, Socket.IO
Validation and tests	Zod request validation; Vitest, Supertest, Socket.IO Client and MongoDB Memory Server
Database	MongoDB instance accessible to the selected backend
Development tools	Git, npm workspace, TypeScript tooling, code editor, browser, Android/iOS tools

Component	Requirement / Technology
	as applicable

1.6.2 Hardware Requirements

Development requires a computer capable of running Node.js 20 or later, npm, a browser, and a local or reachable MongoDB 7 or later instance. Mobile development also requires an Android emulator or device, or a supported iOS development environment. Production capacity should be sized for expected accounts, API traffic, message volume, file storage, and concurrent real-time connections; the repository does not prescribe a fixed server specification.

CHAPTER 2 REQUIREMENTS

2.1 REQUIREMENTS SPECIFICATION

2.1.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow a student to register and authenticate using configured account rules.
FR-02	The system shall allow a user to view and update permitted profile fields and upload a profile image.
FR-03	The system shall allow an authenticated student to create, search, filter, join, and leave communities.
FR-04	The system shall enforce owner, moderator, and member permissions for community management.
FR-05	The system shall provide paginated community message history and real-time chat actions to members.
FR-06	The system shall support authenticated community chat and participant-scoped direct messaging when that extension is enabled.
FR-07	The system shall provide academic resources, metadata, visibility, and download actions according to access rules.
FR-08	The system shall provide notification

ID	Requirement
	views and read-state actions for the signed-in user.
FR-09	The system shall provide administrative views for platform activity, users, communities, resources, and reports.
FR-10	The system shall support student events and peer coordination features represented by the client and service modules.
FR-11	The system shall return structured API responses and validation or authorization errors.
FR-12	The web and mobile clients shall communicate with backend services over configured API and real-time endpoints.

2.1.2 Non-Functional Requirements

Quality	Requirement
Security	Use password hashing, signed access tokens, refresh-token handling, input validation, protected routes, and role checks.
Privacy	Restrict student data and messages to authenticated users with relevant membership or participant permissions.
Reliability	Persist supported messages and records; return consistent API envelopes and handle service errors centrally.
Performance	Paginate lists and message history; avoid returning unbounded collections.
Usability	Provide clear navigation, loading and error states, responsive layouts, and visible interaction feedback.
Maintainability	Separate client, API, service, persistence, validation, and configuration responsibilities.
Portability	Keep backend settings externalized and provide both web and mobile client code.
Scalability	Use modular APIs and storage abstractions; production scaling depends on database, storage, and real-time deployment configuration.

2.2 USER INTERFACES

Screen	Purpose and principal controls
Home / login / registration	Introduces the platform; accepts account credentials and student registration fields.
Dashboard	Summarizes a student's communities and platform activity.
Profile and settings	Displays editable profile information, profile image controls, preferences, and account actions.
Community directory	Supports search, category filtering, community cards, and join actions.
Community details and members	Shows community information, member roster, role labels, and owner/moderator controls.
Community chat	Displays conversation history, compose and attachment controls, reply/edit/delete actions, typing and presence states.
Direct messages	Lists conversations and displays participant-scoped message history and composition controls.
Resources	Lists academic resources, category/search controls, detail metadata, and upload/download actions.
Events and study coordination	Supports event discovery/management and student meeting or scheduling flows present in the client modules.
Notifications	Shows notification history, unread state, mark-read, and bulk actions.
Administration	Provides protected user, community,

Screen	Purpose and principal controls
	resource, report, and platform management pages.

2.3 CONSTRAINTS AND PREREQUISITES

- A reachable MongoDB instance and correctly configured secrets are required for backend operation.
- Browser refresh credentials and mobile token handling follow client-specific authentication paths.
- File uploads require a configured storage provider and permitted file types and sizes.
- Real-time behavior requires a reachable WebSocket or Socket.IO endpoint and compatible proxy settings.
- Some third-party integrations and production providers require credentials and environment variables that are not committed to the repository. Local, test, and production environments should use separate database URLs, signing secrets, upload settings, and allowed-origin configuration.
- The deployed API and Socket.IO endpoints must be reachable by both clients, and reverse-proxy settings must preserve the required HTTP and real-time routes.

CHAPTER 3 ANALYSIS

3.1 EXISTING SYSTEM

In a fragmented workflow, students rely on separate tools for class communication, file exchange, group coordination, event notices, and peer discovery. The separation makes it difficult to keep resources connected to their academic communities and to apply consistent membership and moderation rules. This description is the project problem context rather than a claim that all institutions use the same process.

3.2 PROPOSED SYSTEM

StudyConnect combines student identity, academic communities, messaging, resource sharing, notifications, and administration in one application. Community membership scopes access to group content, while the Express backend checks identity and permissions before completing protected operations. The React web and React Native clients use a shared REST API and Socket.IO channels for supported live updates. The modular design allows features to be implemented and reviewed in manageable milestones.

3.3 USE CASE MODEL

Figure 3.1 Primary use case model

student -> register / sign in / manage profile
student -> discover / join / manage communities
student -> chat / access resources / receive notifications
owner -> manage community and moderator assignments
moderator -> moderate community members and content
administrator -> manage users, reports, and platform content

3.3.1 Actors and Responsibilities

Actor	Responsibilities
Student	Registers, maintains profile, joins communities, communicates, shares or accesses resources, and participates in platform activities.
Community owner	Creates and manages a community and assigns moderators according to the role rules.
Moderator	Supports community oversight and member moderation within assigned authority.
Administrator	Manages platform users and content and reviews reports through protected administration functions.
Email/storage service	External infrastructure used for email delivery and file persistence when configured.

3.4 FLOW OF EVENTS

3.4.1 Student Registration and Sign In

1. The student opens the registration or sign-in screen and enters the required fields.
2. The client sends a validated request to the authentication API.
3. The backend applies configured account rules, checks account state, and verifies credentials.
4. On success, the backend returns an access credential and refresh mechanism appropriate to the client.
5. The client establishes the authenticated session and loads protected profile or dashboard data.

3.4.2 Join a Community and Send a Message

1. The student searches the community directory and opens a community.
2. The student requests membership; the backend verifies the account and applies membership rules.
3. The client loads the member roster and paginated message history when access is granted.
4. The authenticated client joins the community real-time channel.
5. The student sends a message; the backend validates membership and content, persists the message, and broadcasts the resulting event.
6. Other connected members receive the event and update their message view.

3.5 ACTIVITY FLOW

Figure 3.2 Community message activity flow

open community -> authenticate user -> check membership
rejected -> show error
allowed -> compose message -> validate input -> persist message
persisted message -> broadcast to community room -> update connected member views

CHAPTER 4 DESIGN

4.1 SYSTEM ARCHITECTURE

The architecture has four main layers. The React web client and React Native mobile client present student and administrative workflows. Express routes and controllers receive HTTP requests, while middleware handles authentication, authorization, validation, rate limits, and errors. Services apply feature rules and coordinate repositories and providers. Mongoose models and repositories read and write MongoDB documents. Socket.IO supports authorized real-time communication. A storage service isolates local or configured external file storage, and other external services are used only when their environment settings are supplied.

Figure 4.1 Layered application architecture

react web client and react native mobile client
-> express rest api and socket.io gateway
-> route, middleware, controller, service, repository
-> mongoose models and mongodb
-> configured storage, email, and other providers

4.1.1 Backend Responsibilities

Layer	Responsibility
Route / Controller	Maps endpoints or messages to application operations and translates transport input/output.
Validation middleware	Parses and validates incoming request data before feature services run.
Service	Applies business rules, authorization-related decisions, and multi-step workflows.
Repository	Reads and writes domain records using database queries.

Layer	Responsibility
Mongoose model	Defines MongoDB document fields, validation, references, and indexes.
Infrastructure provider	Encapsulates file storage, email, token generation, and real-time messaging concerns.

4.2 DATABASE DESIGN

MongoDB is the document database. The main data areas include users, refresh tokens, communities and memberships, messages, conversations, resources, notifications, reports, and administrative audit records. Mongoose models define document fields, validation rules, indexes, and references; repository functions provide the queries used by services. Fields and indexes can vary by feature and are defined in the backend model and schema code.

Collection / entity	Purpose and key relationships
User	Student or administrator identity, profile, role and account status. Referenced by communities, messages, resources and other user activity.
RefreshToken	Session refresh state, token identity, expiry and revocation metadata linked to a user.
Community	Academic group metadata, visibility, owner, moderators and denormalized member count.
CommunityMember	User-to-community membership and role such as OWNER, MODERATOR or MEMBER.
Message	Community message content, sender,

Collection / entity	Purpose and key relationships
	attachments, replies, edit/delete state and timestamps.
Conversation / DirectMessage	Private conversation participants and persisted participant messages.
Resource	Academic file metadata, visibility, category, uploader and community association.
Notification	Recipient, event type, related record and read state.
Report / AdminLog	Moderation target and review state; administrative action record.

4.3 ENTITY RELATIONSHIP MODEL

Figure 4.2 Core entity relationship model

user 1 -> many refresh tokens
user many -> many communities through community member
user 1 -> many messages; each message belongs to a community
conversation 1 -> many direct messages with participant users
user 1 -> many resources, notifications, and reports

Figure 4.2 presents the main data relationships conceptually. Related records are connected with MongoDB identifiers and Mongoose references rather than relational foreign-key constraints. Unique indexes protect fields such as account identifiers and community slugs, while selected counters may be stored with their owning documents for efficient display. The backend model and repository code define the actual fields and query behavior.

4.4 SEQUENCE DESIGN

Figure 4.3 Community message sequence

student -> client: compose and send message
client -> api service: validate request
api service -> database: check membership and persist message
api service -> community room: broadcast event
community room -> connected clients: deliver event

4.5 CLASS AND RESPONSIBILITY DESIGN

Figure 4.4 Backend class responsibility model

Express route -> authentication and validation middleware -> controller
controller -> feature service -> repository -> Mongoose model -> MongoDB
service -> token, storage, email, and Socket.IO providers
React page -> hook / query -> API client -> REST or Socket.IO

4.6 API AND SECURITY DESIGN

Protected HTTP operations use bearer access tokens. Web authentication uses refresh-cookie behavior; the mobile path is distinguished through a client-platform header and secure client storage. Password hashes are not returned through normal user-facing responses. Authorization is enforced through account state and role/membership checks. APIs use a common success envelope and structured error handling. Paginated endpoints limit the amount of data returned in one request.

Role	Typical permissions
STUDENT	Manage own profile and participate in allowed communities and platform features.
OWNER	Manage own community, its settings and moderator assignments.
MODERATOR	Moderate members or content within assigned community authority.

Role	Typical permissions
ADMIN	Use platform administration functions for user and content oversight.

CHAPTER 5 IMPLEMENTATION

5.1 TECHNOLOGY STACK

Layer	Technologies present in the repository
Web	React 19, TypeScript, Vite, React Router 7, Tailwind CSS, Zustand, TanStack Query, Axios, Socket.IO Client and selected real-time integrations.
Mobile	React Native 0.86, React Navigation 7, TanStack Query, Axios, Zustand, secure storage packages, Socket.IO Client.
Express backend	Node.js, TypeScript, Express, Mongoose, MongoDB, Socket.IO, Zod validation and Vitest-based tests.
Validation and testing	Zod request schemas, Vitest, Supertest, Socket.IO Client and MongoDB Memory Server.
Data and infrastructure	MongoDB, local/file storage abstractions and optional configured external providers.

5.2 MODULES

Authentication and account security

Registration, login, logout, refresh-token handling, password change and reset workflows, profile access, and access control.

Student profiles

Profile retrieval and editing, profile images, account settings and student-facing dashboard data.

Communities

Community creation and discovery, categories, visibility, membership, member listings, moderator assignment and member removal.

Community chat

Persisted history, message create/edit/delete, attachments, reply references, real-time delivery, typing state and presence.

Direct messaging extension

Participant-scoped conversation workflows are a planned extension; the current Express route list does not expose a direct-message API.

Resource sharing

Community and broader resource listings, metadata, upload, detail, visibility, category filtering and download tracking.

Notifications

Recipient-scoped notification listing, unread counts, read actions and real-time notification events.

Administration and moderation

Protected dashboards and management routes for users, communities, resources, reports and moderation records.

Events and study coordination

Client and API areas for campus events, connections, meetings, scheduling, polls and related study workflows, subject to configured services and deployment.

Web and mobile clients

Responsive browser application and mobile application that provide student and administrative navigation and feature screens.

5.3 IMPLEMENTATION DETAILS

The web client organizes route pages, reusable components, API modules, hooks, state stores, and shared types. The mobile client organizes navigation, screens, API access, and secure credential storage. Both clients send requests to the Express service. On the server, routes attach middleware and controllers; Zod validators check input; services apply business rules; repositories perform MongoDB queries through Mongoose models. Authentication middleware validates access tokens, role and membership checks protect operations, and the error middleware returns a consistent response structure. Socket.IO gateways authenticate connections, check community membership, and publish authorized chat events.

Uploads pass through the backend storage service so profile images, community banners, chat attachments, and resources can use a common interface. The local provider supports development, while external providers require deployment configuration. Community chat uses authenticated Socket.IO connections and verifies membership before a user joins a room or sends an event. REST endpoints return paginated message history and handle attachment uploads. API routes, request fields, and socket event names are documented with the related backend features.

5.4 CONFIGURATION AND DEPLOYMENT

1. Install Node.js 20 or later, MongoDB 7 or later, and the npm workspace dependencies. Copy the backend and client environment templates before starting the applications.

2. Configure the MongoDB connection string, distinct random access/refresh signing secrets, allowed email domains, and secure cookie settings for HTTPS.

3. Set API and Socket.IO origins, CORS/allowed-origin rules, upload directory or storage provider settings, and email credentials when those services are enabled.

4. Use the workspace scripts to build the TypeScript backend and web client. Run the backend service and serve the generated web assets through the selected hosting provider or reverse proxy.

5. Use local file storage for development or a single instance. Configure shared object storage and a suitable Socket.IO adapter before running multiple backend instances.

6. Before release, verify health and readiness endpoints, HTTPS and proxy routing, database connectivity, upload permissions, secrets, and compatibility with the web and mobile clients. Back up the database and define a rollback plan for application configuration changes.

CHAPTER 6 USER SCREENS

The screens below describe the principal page groups represented by the web and mobile source. Exact labels and visible actions may vary by role, device size, feature configuration, and current deployment.

6.1 NAVIGATION

The web client separates public authentication routes from protected student and administration areas. The student experience centers on the dashboard, communities, chat, resources, events, connections, notifications, and profile settings. Administration routes are protected by elevated authorization. The mobile application uses native stack and tab navigation for core student tasks.

6.2 STUDENT SCREENS

Screen group	Content and user interaction
Authentication	Sign in, registration, forgot password, reset password and validation feedback.
Dashboard	Overview of communities and recent student activity.
Communities	Search, category filters, community creation, join/leave, details and member list.
Community chat	Message list, pagination, compose, reply, attachment, edit/delete, typing and online state.
Messaging extensions	Reserved interface area for participant-scoped conversations; enable only after an API and authorization flow are available.
Resources	Resource discovery, category/search

Screen group	Content and user interaction
	filters, upload form, details and download.
Events and study tools	Event listings and management, meeting and schedule interactions in available feature flows.
Connections and profile	Student profile details, preferences, connections and profile image controls.
Notifications	Notification list, unread count, mark read and clear actions.

6.3 ADMINISTRATION SCREENS

Screen	Administrative task
Admin dashboard	Review platform summary information and activity.
User management	Find accounts and apply supported account status or role actions.
Community and resource management	Review or remove content under administrator permissions.
Reports and moderation	Review submitted reports and record moderation outcomes.

6.4 MOBILE SCREENS

The mobile client is designed around native navigation and touch-sized interactions. It includes API configuration, authentication integration, student-facing navigation and service modules for realtime and secure credential handling. Mobile screen details should be validated against the built application on the target device before a release.

CHAPTER 7 TESTING METHODOLOGY

7.1 TESTING STRATEGY

The backend test suite is configured with Vitest, Supertest, Socket.IO Client, and MongoDB Memory Server. Tests should cover service rules, REST validation and responses, persistence, authorization, upload handling, real-time room access, and error conditions. Web tests use Vitest, Testing Library, and JSDOM. A complete release review should also check responsive layouts and representative mobile flows. The scenarios below document expected behavior; no test run was performed while revising this report.

7.2 REPRESENTATIVE TEST SCENARIOS

ID	Scenario	Expected result
TC-01	Register with valid account data	Account is created and authentication response follows the API contract.
TC-02	Authenticate with invalid credentials	Request is rejected without disclosing sensitive account data.
TC-03	Update another user's profile	Request is denied by identity and authorization checks.
TC-04	Join a community twice	Membership remains consistent and duplicate membership is handled safely.
TC-05	Non-member reads private community messages	Request or realtime room join is rejected.
TC-06	Member sends a valid	Message is persisted and

ID	Scenario	Expected result
	message	delivered to authorized room participants.
TC-07	Unauthenticated socket attempts to join a community room	Connection or room join is rejected before the user can read or send messages.
TC-08	Upload an unsupported or oversized file	Validation rejects the upload and no invalid resource is exposed.
TC-09	Student calls an administrator endpoint	Request is denied.
TC-10	Refresh a session token	Valid refresh is rotated; invalid or revoked token is rejected.
TC-11	Mark a notification as read	Only the recipient's notification state is updated.
TC-12	Load long resource or message history	Results are paginated and response size remains bounded.

7.3 VERIFICATION STATUS

The repository contains backend integration tests for community creation, unique community names, membership, member listing, owner-only moderator promotion, REST message history, membership enforcement, and Socket.IO broadcasts. It also includes security, notification, and client-side tests. These test files document the available verification coverage; no additional test suite was executed while preparing this report, so this document does not claim a new passing test run.

CHAPTER 8 CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

StudyConnect brings student identity, academic communities, communication, resource exchange, notifications, and administration into one web and mobile platform. Its TypeScript and Node.js Express backend provides REST services, MongoDB persistence through Mongoose, and Socket.IO communication for supported live features. Authentication, role checks, and community membership rules protect student and administrative workflows. The report covers the project structure, requirements, analysis, design, implementation, screens, and verification approach.

The system depends on correct environment settings, database availability, secure secrets, and configured storage or email providers. Source code and test coverage describe the implementation, but do not by themselves establish a production deployment or an institutional rollout. The modular client and service design provides a foundation for continued feature validation, operational hardening, and controlled deployment.

8.2 FUTURE ENHANCEMENTS

- Complete Express-service production readiness with load testing, monitoring, rollback procedures, and database backup and recovery.
- Expand end-to-end tests across authentication, communities, chat, uploads, notifications, and administration; validate mobile parity and reconnection behavior.
- Add institutional onboarding and identity controls as needed, and review accessibility with students and assistive technologies.
- Improve logs, metrics, and alerts; use secure shared storage, define retention and moderation policies, and tune MongoDB indexes and pagination under load.

CHAPTER 9 REFERENCES

[1] StudyConnect Development Team, “StudyConnect Phase 3 Community Chat,” docs/COMMUNITY_CHAT.md, internal project documentation, 2026.

[2] StudyConnect Development Team, StudyConnect backend package manifest (backend/package.json) and implementation source (backend/src/), internal project repository, 2026.

[3] StudyConnect Development Team, StudyConnect backend tests, backend/tests/, internal project repository, 2026.

[4] StudyConnect Development Team, StudyConnect web and React Native client source and package manifests (web/package.json and mobile/package.json), internal project repository, 2026.

[5] “Guidelines to Prepare B.Tech Project Documentation,” user-supplied PDF, 2016–2017 format reference.

[6] Nangunoory Abhinav, Parunandi Anunaya, Pulapalli Murali Krishna, and Greeshmanth, “Federated Hyper Dimensional Computing: Comprehensive Analysis and Robust Communication,” Mini Project Report, Department of Computer Science and Engineering, Anurag University, 2025–2026. User-supplied sample report.

[7] Node.js Project, “Node.js v20 API Documentation.” [Online]. Available: https://nodejs.org/download/release/latest-v20.x/docs/api/. [Accessed: 12-Oct-2026].

[8] Express.js, “Routing,” Express 5.x Documentation. [Online]. Available: https://expressjs.com/en/5x/guide/routing/. [Accessed: 12-Oct-2026].

[9] Microsoft, “The TypeScript Handbook.” [Online]. Available: https://www.typescriptlang.org/docs/handbook/intro.html. [Accessed: 12-Oct-2026].

[10] Meta Open Source, “Quick Start,” React Documentation. [Online]. Available: https://react.dev/learn. [Accessed: 12-Oct-2026].

[11] Meta Open Source, “Getting Started,” React Native 0.86 Documentation. [Online]. Available: https://reactnative.dev/docs/0.86/getting-started. [Accessed: 12-Oct-2026].

[12] MongoDB, Inc., “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/manual/. [Accessed: 12-Oct-2026].

[13] Automattic, “Schemas,” Mongoose 8.x Documentation. [Online]. Available: https://mongoosejs.com/docs/8.x/docs/guide.html. [Accessed: 12-Oct-2026].

[14] Socket.IO, “Introduction,” Socket.IO 4.x Documentation. [Online]. Available: https://socket.io/docs/v4/. [Accessed: 12-Oct-2026].

[15] Zod Project, “Basic Usage,” Zod 3 Documentation. [Online]. Available: https://v3.zod.dev/. [Accessed: 12-Oct-2026].

[16] Vite Project, “Getting Started.” [Online]. Available: https://vite.dev/guide/. [Accessed: 12-Oct-2026].

[17] Vitest Project, “Getting Started.” [Online]. Available: https://vitest.dev/guide/. [Accessed: 12-Oct-2026].

[18] Forward Email and contributors, “Supertest: Testing Node.js HTTP Servers.” [Online]. Available: https://github.com/ladjs/supertest. [Accessed: 12-Oct-2026].

[19] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[20] OWASP Foundation, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: 12-Oct-2026].

[21] Object Management Group, “Unified Modeling Language (UML), Version 2.5.1,” December 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1. [Accessed: 12-Oct-2026].